

Agentic Tester App

Gemini-powered · Agent A → Agent B → Agent C (max 5 review loops)

Step 1 — Gemini API key

Enter your Gemini API key



.....



API key entered ✓

Step 2 — Upload requirements PDF

Choose a PDF file



Drag and drop file here
Limit 200MB per file • PDF

Browse files



Capstone requirements document.pdf 230.0KB



PDF loaded: **Capstone requirements document.pdf** — 44,663 characters extracted ✓

Step 3 — Run agents



Run all agents

```
▶ AGENT B – Generate Playwright tests (v1)
=====
☹ Agent B: Generating Playwright tests (version 1)...
✅ Agent B: Generated 102 test functions (v1).

=====
▶ AGENT C – Review loop (max 5 attempts)
=====
🔍 Agent C: Reviewing tests (attempt 1/5)...
Verdict: FAIL | Score: 66/100
🔄 Sending critique to Agent B for rewrite (v2)...
☹ Agent B: Generating Playwright tests (version 2)...
✅ Agent B: Generated 102 test functions (v2).
🔍 Agent C: Reviewing tests (attempt 2/5)...
Verdict: PASS | Score: 90/100
✅ Agent C: Tests passed on attempt 2.

=====
🏁 Done – Verdict: PASS | Attempts: 2
=====
```

Results

Verdict	Score	Attempts	Requirements
PASS	90/100	2/5	102

-  Final tests
-  Requirements
-  Review report

```
# Generated: 2026-03-23 23:28:47
# Version: 2

import pytest
from playwright.sync_api import Page, expect, sync_playwright
import os
import time
```

```

BASE_URL = "https://the-internet.herokuapp.com"

def test_FR_G_01_Home_Page_Examples_List(page: Page):
    """FR-G-01: The home page must display a list of links to all available example
    page.goto(BASE_URL)
    expect(page.get_by_role("heading", name="Welcome to the-internet")).to_be_visible()
    expect(page.get_by_role("heading", name="Available Examples")).to_be_visible()

    # Get all links under 'Available Examples'
    links = page.locator("#content ul li a").all()

    # Assert visibility for a representative sample (e.g., first 15 links)
    # or all links if the list is not excessively long.
    # For this site, iterating all is feasible and robust.
    assert len(links) > 0, "No example links found on the home page."

    # Iterate through all links and assert their visibility and text content
    for link in links:
        expect(link).to_be_visible()
        # The link text should match the example name
        link_text = link.text_content().strip()
        assert len(link_text) > 0, "Link text should not be empty"
        # Further specific checks could be added here if there's a known list of e

def test_FR_G_02_Footer_Attribution(page: Page):
    """FR-G-02: Most example pages must display a consistent footer section attrib
    page.goto(f"{BASE_URL}/checkboxes")
    expect(page.locator("#page-footer")).to_be_visible()
    expect(page.locator("#page-footer")).to_contain_text("Powered by Elemental Sel

def test_FR_CB_01_Checkboxes(page: Page):
    """FR-CB-01: The checkboxes page must display two checkboxes with labels."""
    page.goto(f"{BASE_URL}/checkboxes")
    expect(page.get_by_role("heading", name="Checkboxes")).to_be_visible()
    checkboxes = page.locator("input[type='checkbox']").all()
    assert len(checkboxes) >= 2, "Expected at least two checkboxes"

    # Assert labels or nearby text for the checkboxes
    expect(page.locator("text=checkbox 1")).to_be_visible()
    expect(page.locator("text=checkbox 2")).to_be_visible()

def test_FR_CB_02_Checkboxes(page: Page):
    """FR-CB-02: The user must be able to toggle checkbox states by clicking them.

```

```
page.goto(f"{BASE_URL}/checkboxes")
checkbox1 = page.locator("form#checkboxes input").first
checkbox2 = page.locator("form#checkboxes input").last

# Check initial state and toggle
initial_state_1 = checkbox1.is_checked()
checkbox1.click()
expect(checkbox1).to_be_checked(checked=not initial_state_1)

initial_state_2 = checkbox2.is_checked()
checkbox2.click()
expect(checkbox2).to_be_checked(checked=not initial_state_2)

def test_FR_FA_01_Form_Authentication_Login(page: Page):
    """FR-FA-01: The login page must display username and password fields with a L
    page.goto(f"{BASE_URL}/login")
    expect(page.get_by_role("heading", name="Login Page")).to_be_visible()
    expect(page.get_by_label("Username")).to_be_visible()
    expect(page.get_by_label("Password")).to_be_visible()
    expect(page.get_by_role("button", name="Login")).to_be_visible()

def test_FR_FA_02_Form_Authentication_Login(page: Page):
    """FR-FA-02: Valid credentials ('tomsmith', 'SuperSecretPassword!') must grant
    page.goto(f"{BASE_URL}/login")
    page.get_by_label("Username").fill("tomsmith")
    page.get_by_label("Password").fill("SuperSecretPassword!")
    page.get_by_role("button", name="Login").click()
    expect(page).to_have_url(f"{BASE_URL}/secure")
    expect(page.get_by_text("You logged into a secure area!")).to_be_visible()
    expect(page.locator("#flash.success")).to_be_visible()

def test_FR_FA_03_Form_Authentication_Login(page: Page):
    """FR-FA-03: Incorrect login information must display an error message and pre
    page.goto(f"{BASE_URL}/login")
    page.get_by_label("Username").fill("wronguser")
    page.get_by_label("Password").fill("wrongpass")
    page.get_by_role("button", name="Login").click()
    expect(page).to_have_url(f"{BASE_URL}/login")
    expect(page.locator("#flash.error")).to_be_visible()
    expect(page.locator("#flash.error")).to_contain_text("Your username is invalid

def test_FR_DD_01_Dropdown_List(page: Page):
    """FR-DD-01: The dropdown page must display a selectable dropdown list with tw
```

```

page.goto(f"{BASE_URL}/dropdown")
expect(page.get_by_role("heading", name="Dropdown List")).to_be_visible()
dropdown = page.locator("#dropdown")
expect(dropdown).to_be_visible()
expect(dropdown.locator("option[selected='selected']")).to_have_text("Please s
expect(dropdown.locator("option[value='1']")).to_have_text("Option 1")
expect(dropdown.locator("option[value='2']")).to_have_text("Option 2")

def test_FR_DD_02_Dropdown_List(page: Page):
    """FR-DD-02: The user must be able to change the dropdown selection."""
    page.goto(f"{BASE_URL}/dropdown")
    dropdown = page.locator("#dropdown")
    dropdown.select_option("1")
    expect(dropdown).to_have_value("1")
    dropdown.select_option("2")
    expect(dropdown).to_have_value("2")

def test_FR_DC_01_Dynamic_Controls(page: Page):
    """FR-DC-01: The page must demonstrate asynchronous changes to elements like c
    page.goto(f"{BASE_URL}/dynamic_controls")
    expect(page.get_by_role("heading", name="Dynamic Controls")).to_be_visible()
    expect(page.get_by_text("This example demonstrates the use of AJAX in updating
    expect(page.locator("#checkbox")).to_be_visible()
    expect(page.get_by_role("button", name="Remove")).to_be_visible()
    expect(page.locator("#text-example input")).to_be_disabled()
    expect(page.get_by_role("button", name="Enable")).to_be_visible()

def test_FR_DL_01_Dynamic_Loading(page: Page):
    """FR-DL-01: The page must present two separate examples of dynamically loaded
    page.goto(f"{BASE_URL}/dynamic_loading")
    expect(page.get_by_role("heading", name="Dynamically Loaded Page Elements")).t
    expect(page.get_by_role("link", name="Example 1: Element on page that is hidde
    expect(page.get_by_role("link", name="Example 2: Element rendered after the fa

def test_FR_UP_01_File_Upload(page: Page):
    """FR-UP-01: The page must provide a file selection input, an upload button, a
    page.goto(f"{BASE_URL}/upload")
    expect(page.get_by_role("heading", name="File Uploader")).to_be_visible()
    expect(page.locator("#file-upload")).to_be_visible() # File input
    expect(page.locator("#file-submit")).to_be_visible() # Upload button
    expect(page.locator("#drag-and-drop-upload")).to_be_visible() # Drag and drop

def test_FR_UP_02_File_Upload(page: Page, tmp_path):

```

```
"""FR-UP-02: The system must accept a selected file upon clicking the 'Upload'
page.goto(f"{BASE_URL}/upload")

# Create a dummy file dynamically using tmp_path
file_content = "This is a test file."
file_name = "test_file.txt"
file_path = tmp_path / file_name
file_path.write_text(file_content)

page.locator("#file-upload").set_input_files(str(file_path))
page.locator("#file-submit").click()

expect(page.get_by_role("heading", name="File Uploaded!")).to_be_visible()
expect(page.locator("#uploaded-files")).to_contain_text(file_name)

def test_FR_JA_01_JavaScript_Alerts(page: Page):
    """FR-JA-01: The page must present buttons that trigger different types of JavaScript alerts"""
    page.goto(f"{BASE_URL}/javascript_alerts")
    expect(page.get_by_role("heading", name="JavaScript Alerts")).to_be_visible()
    expect(page.get_by_role("button", name="Click for JS Alert")).to_be_visible()
    expect(page.get_by_role("button", name="Click for JS Confirm")).to_be_visible()
    expect(page.get_by_role("button", name="Click for JS Prompt")).to_be_visible()
    expect(page.get_by_text("Result:")).to_be_visible()

def test_FR_JA_02_JavaScript_Alerts(page: Page):
    """FR-JA-02: Clicking 'Click for JS Alert' must trigger a simple JavaScript alert"""
    page.goto(f"{BASE_URL}/javascript_alerts")
    page.on("dialog", lambda dialog: dialog.accept())
    page.get_by_role("button", name="Click for JS Alert").click()
    expect(page.locator("#result")).to_have_text("You successfully clicked an alert")

def test_FR_JA_03_JavaScript_Alerts(page: Page):
    """FR-JA-03: Clicking 'Click for JS Confirm' must display a confirm dialog, and return 'Ok'"""
    page.goto(f"{BASE_URL}/javascript_alerts")

    # Test JS Confirm
    page.on("dialog", lambda dialog: dialog.accept())
    page.get_by_role("button", name="Click for JS Confirm").click()
    expect(page.locator("#result")).to_have_text("You clicked: Ok")

    # Test JS Prompt
    page.on("dialog", lambda dialog: dialog.accept("Playwright Test"))
    page.get_by_role("button", name="Click for JS Prompt").click()
```

```
expect(page.locator("#result")).to_have_text("You entered: Playwright Test")

def test_FR_DDP_01_Drag_and_Drop(page: Page):
    """FR-DDP-01: The page must display a region that visually suggests drag-and-drop"""
    page.goto(f"{BASE_URL}/drag_and_drop")
    expect(page.get_by_role("heading", name="Drag and Drop")).to_be_visible()
    expect(page.locator("#column-a")).to_be_visible()
    expect(page.locator("#column-b")).to_be_visible()

def test_FR_TB_01_Sortable_Data_Tables(page: Page):
    """FR-TB-01: The page must display at least two table examples with specific content"""
    page.goto(f"{BASE_URL}/tables")
    expect(page.get_by_role("heading", name="Data Tables")).to_be_visible()
    expect(page.get_by_text("Example 1")).to_be_visible()
    expect(page.get_by_text("Example 2")).to_be_visible()

    # Check headers for table 1
    table1_headers = page.locator("#table1 th").all_text_contents()
    expected_headers = ["Last Name", "First Name", "Email", "Due", "Web Site", "Actions"]
    assert all(header in table1_headers for header in expected_headers)

    # Check for rows in table 1
    expect(page.locator("#table1 tbody tr")).to_have_count(4)

def test_FR_TB_02_Sortable_Data_Tables(page: Page):
    """FR-TB-02: Each table row must include 'edit' and 'delete' action controls."""
    page.goto(f"{BASE_URL}/tables")
    first_row_actions = page.locator("#table1 tbody tr").first.locator("td:last-child")
    assert "edit" in first_row_actions
    assert "delete" in first_row_actions

def test_FR_NM_01_Notification_Messages(page: Page):
    """FR-NM-01: The page must display a notification message above the heading."""
    page.goto(f"{BASE_URL}/notification_message_rendered")
    expect(page.get_by_role("heading", name="Notification Message")).to_be_visible()
    flash_message = page.locator("#flash")
    expect(flash_message).to_be_visible()
    expect(flash_message).not_to_be_empty()
    expect(flash_message).to_contain_text("Action") # Less specific assertion

def test_FR_NM_02_Notification_Messages(page: Page):
    """FR-NM-02: Clicking the 'Click here to load a new message' link must display a new message"""
    page.goto(f"{BASE_URL}/notification_message_rendered")
```

```

initial_message = page.locator("#flash").text_content()
page.get_by_role("link", name="Click here").click()
expect(page.locator("#flash")).not_to_have_text(initial_message) # Message sho

def test_FR_EA_01_Entry_Ad(page: Page):
    """FR-EA-01: An advertisement modal must automatically display when the page l
    page.goto(f"{BASE_URL}/entry_ad")
    modal = page.locator(".modal")
    expect(modal).to_be_visible()
    expect(modal.get_by_role("heading", name="THIS IS A MODAL WINDOW")).to_be_visi

def test_FR_EA_02_Entry_Ad(page: Page):
    """FR-EA-02: Closing the ad must dismiss it and prevent its reappearance on su
    page.goto(f"{BASE_URL}/entry_ad")
    modal = page.locator(".modal")
    expect(modal).to_be_visible()
    modal.get_by_text("Close").click()
    expect(modal).not_to_be_visible()

    page.reload()
    expect(modal).not_to_be_visible()

def test_FR_EA_03_Entry_Ad(page: Page):
    """FR-EA-03: The user must be able to re-enable the ad."""
    page.goto(f"{BASE_URL}/entry_ad")
    modal = page.locator(".modal")
    expect(modal).to_be_visible()
    modal.get_by_text("Close").click()
    expect(modal).not_to_be_visible()

    page.get_by_role("link", name="click here").click()
    page.reload()
    expect(modal).to_be_visible()

def test_FR_TY_01_Typos(page: Page):
    """FR-TY-01: The page must randomly include or exclude a typo in the displayed
    page.goto(f"{BASE_URL}/typos")
    expect(page.get_by_role("heading", name="Typos")).to_be_visible()
    paragraph = page.locator(".example p").nth(1)

    # Corrected assertion: check for the presence of the correct text AND the spec
    # This test acknowledges the randomness and checks for either variant.
    # The page content is either "Sometimes you and I get a typo." or "Sometimes y

```



```

# We need to check if the text contains either the correct version or the typo
# Since the requirement states "randomly include or exclude a typo", we can't
# We assert that it contains *one of* the expected texts.
# For a single test run, we can only check what's currently there.
# To fully cover the "randomly include or exclude" aspect, multiple runs or a
# For this single test, we assert it contains the base text and then check for

# Get the text content
text_content = paragraph.text_content()

# Assert that the base part of the sentence is always present
expect(paragraph).to_contain_text("Sometimes you and I")

# Assert that it contains either the correct ending or the typo ending
# This is a functional test, so we expect it to be one of the valid states.
assert "get a typo." in text_content or "won,t get a typo." in text_content, \
    f"Paragraph text did not contain expected correct or typo text: '{text_con

def test_FR_ARE_01_Add_Remove_Elements(page: Page):
    """FR-ARE-01: The page must render a button for dynamically adding elements."""
    page.goto(f"{BASE_URL}/add_remove_elements/")
    expect(page.get_by_role("heading", name="Add/Remove Elements")).to_be_visible()
    expect(page.get_by_role("button", name="Add Element")).to_be_visible()

def test_FR_ARE_02_Add_Remove_Elements(page: Page):
    """FR-ARE-02: Clicking 'Add Element' must add a new element to the page."""
    page.goto(f"{BASE_URL}/add_remove_elements/")
    add_button = page.get_by_role("button", name="Add Element")
    add_button.click()
    expect(page.get_by_role("button", name="Delete")).to_have_count(1)

def test_FR_ARE_03_Add_Remove_Elements(page: Page):
    """FR-ARE-03: Dynamically added elements must be removable by clicking their a
    page.goto(f"{BASE_URL}/add_remove_elements/")
    add_button = page.get_by_role("button", name="Add Element")
    add_button.click()
    delete_button = page.get_by_role("button", name="Delete")
    expect(delete_button).to_be_visible()
    delete_button.click()
    expect(delete_button).not_to_be_visible()

def test_FR_DE_01_Disappearing_Elements(page: Page):
    """FR-DE-01: On each page load, certain page elements may appear or disappear.

```

```

page.goto(f"{BASE_URL}/disappearing_elements")
expect(page.get_by_role("heading", name="Disappearing Elements")).to_be_visible()
# Acknowledging limitation: To fully cover 'may appear or disappear',
# the test should ideally reload the page multiple times (e.g., in a loop)
# and assert that the 'Gallery' link is *sometimes* present and *sometimes* absent
# For a single test, we can only check its state on one load.
# We'll assert that the Home link is always there, and Gallery is sometimes there
expect(page.get_by_role("link", name="Home")).to_be_visible()
# We can't assert it's always visible or always hidden due to randomness.
# We can check if it's present on this specific load.
# For a functional test, we expect it to be one of the valid states.
# The presence of the element is the "functional" aspect, its disappearance is not
# So, we assert that if it exists, it's visible.
gallery_link = page.get_by_role("link", name="Gallery")
if gallery_link.count() > 0:
    expect(gallery_link).to_be_visible()
else:
    # If it's not present, that's also a valid outcome for this functional requirement
    # We just can't assert its presence.
    pass # No assertion needed if it's not there, as per the "may appear or disappear" requirement

def test_FR_HV_01_Hovers(page: Page):
    """FR-HV-01: The page must display multiple user images with hidden details."""
    page.goto(f"{BASE_URL}/hovers")
    expect(page.get_by_role("heading", name="Hovers")).to_be_visible()
    images = page.locator(".figure").all()
    assert len(images) >= 3, "Expected at least three user images"
    for i in range(1, 4):
        expect(page.locator(f".figure:nth-child({i}) img")).to_be_visible()
        expect(page.locator(f".figure:nth-child({i}) .figcaption h5")).not_to_be_visible()

def test_FR_HV_02_Hovers(page: Page):
    """FR-HV-02: User details must be revealed when hovering over each image and hovering over the details must reveal the details"""
    page.goto(f"{BASE_URL}/hovers")
    figure = page.locator(".figure").first
    figcaption = figure.locator(".figcaption")

    expect(figcaption).not_to_be_visible()
    figure.hover()
    expect(figcaption).to_be_visible()
    expect(figcaption.get_by_text("name: user1")).to_be_visible()
    expect(figcaption.get_by_role("link", name="View profile")).to_be_visible()

```

```

# Move mouse away (e.g., to the body)
page.locator("body").hover()
expect(figcaption).not_to_be_visible()

def test_FR_AB_01_A_B_Test(page: Page):
    """FR-AB-01: The A/B Test page must describe A/B testing or split testing."""
    page.goto(f"{BASE_URL}/abtest")
    # The heading can be "A/B Test Control" or "A/B Test Variation 1" or "No A/B T
    expect(page.locator("h3")).to_be_visible()
    expect(page.locator(".example")).to_contain_text("Also known as split testing.

def test_FR_AB_02_A_B_Test(page: Page):
    """FR-AB-02: The A/B Test page content may vary, potentially showing 'No A/B T
    page.goto(f"{BASE_URL}/abtest")
    # This test acknowledges the variability. We can't assert a specific heading,
    # but we can assert that one of the expected headings is present.
    heading_text = page.locator("h3").text_content()
    assert heading_text in ["A/B Test Control", "A/B Test Variation 1", "No A/B Te

def test_FR_DCNT_01_Dynamic_Content(page: Page):
    """FR-DCNT-01: The Dynamic Content page must load new text and images on each
    page.goto(f"{BASE_URL}/dynamic_content")
    initial_content = page.locator("#content").inner_html()
    page.reload()
    # Use assert with inner_html for correct comparison
    assert page.locator("#content").inner_html() != initial_content, "Content shou

def test_FR_DCNT_02_Dynamic_Content(page: Page):
    """FR-DCNT-02: Appending `?with_content=static` to the URL or using the 'click
    page.goto(f"{BASE_URL}/dynamic_content?with_content=static")
    initial_content = page.locator("#content").inner_html()
    page.reload()
    # Use assert with inner_html for correct comparison
    assert page.locator("#content").inner_html() == initial_content, "Content shou

def test_FR_SC_01_Status_Codes(page: Page):
    """FR-SC-01: The Status Codes page must explain HTTP status codes and list sta
    page.goto(f"{BASE_URL}/status_codes")
    expect(page.get_by_role("heading", name="Status Codes")).to_be_visible()
    expect(page.locator(".example")).to_contain_text("HTTP status codes are a stan
    expect(page.get_by_role("link", name="200")).to_be_visible()
    expect(page.get_by_role("link", name="301")).to_be_visible()
    expect(page.get_by_role("link", name="404")).to_be_visible()

```

```
expect(page.get_by_role("link", name="500")).to_be_visible()
expect(page.get_by_role("link", name="here")).to_be_visible() # Link to comple

def test_FR_IN_01_Inputs(page: Page):
    """FR-IN-01: The Inputs page must render a numeric input field labeled 'Number'
    page.goto(f"{BASE_URL}/inputs")
    expect(page.get_by_role("heading", name="Inputs")).to_be_visible()
    expect(page.locator("input[type='number']").to_be_visible()
    expect(page.locator(".example")).to_contain_text("Number") # Label is usually

def test_FR_IN_02_Inputs(page: Page):
    """FR-IN-02: The numeric input field must accept numeric entry according to br
    page.goto(f"{BASE_URL}/inputs")
    input_field = page.locator("input[type='number']")
    input_field.fill("123")
    expect(input_field).to_have_value("123")

    # Test up/down arrows (simulating key presses)
    input_field.press("ArrowUp")
    expect(input_field).to_have_value("124")
    input_field.press("ArrowDown")
    expect(input_field).to_have_value("123")

def test_FR_HS_01_Horizontal_Slider(page: Page):
    """FR-HS-01: The Horizontal Slider page must show a horizontal slider control
    page.goto(f"{BASE_URL}/horizontal_slider")
    expect(page.get_by_role("heading", name="Horizontal Slider")).to_be_visible()
    expect(page.locator("input[type='range']").to_be_visible() # Slider control
    expect(page.locator("#range")).to_be_visible() # Value indicator

def test_FR_HS_02_Horizontal_Slider(page: Page):
    """FR-HS-02: Moving the slider must update the displayed value."""
    page.goto(f"{BASE_URL}/horizontal_slider")
    slider = page.locator("input[type='range']")
    value_indicator = page.locator("#range")

    # Set slider to a specific value
    slider.fill("3.5")
    expect(value_indicator).to_have_text("3.5")

    # Move slider using keyboard (e.g., ArrowRight)
    slider.press("ArrowRight")
    expect(value_indicator).to_have_text("4")
```

```

def test_FR_CM_01_Context_Menu(page: Page):
    """FR-CM-01: The Context Menu page must display a box area where right-click t
    page.goto(f"{BASE_URL}/context_menu")
    expect(page.get_by_role("heading", name="Context Menu")).to_be_visible()
    hot_spot = page.locator("#hot-spot")
    expect(hot_spot).to_be_visible()

    # Acknowledge that directly asserting the custom menu item is difficult.
    # Playwright's `page.on("dialog")` can catch the alert, but not inspect the co
    # For better coverage, consider a visual regression test or a JavaScript injec
    # to inspect the context menu DOM if possible (which is complex and often brow
    # For this test, we'll assert the alert is triggered, which implies the menu i
    # The requirement is about the menu *appearing* with the item.
    # We can simulate the right-click and expect an alert to be triggered.
    page.on("dialog", lambda dialog: dialog.accept())
    hot_spot.click(button="right")
    # If an alert is triggered, it implies the custom context menu was invoked and
    # The absence of a default browser context menu (which would not trigger an al
    # We can't directly assert the text "the-internet" in the context menu itself.
    # The next test FR-CM-02 covers the alert.

def test_FR_CM_02_Context_Menu(page: Page):
    """FR-CM-02: Selecting the custom context menu item must trigger a JavaScript
    page.goto(f"{BASE_URL}/context_menu")
    hot_spot = page.locator("#hot-spot")

    # Expect a dialog to be opened and accept it
    page.on("dialog", lambda dialog: dialog.accept())
    hot_spot.click(button="right")
    # The assertion is implicit in the dialog handler. If no dialog appears, the t
    # We can add a simple assertion to ensure the page is still there and function
    expect(page.get_by_role("heading", name="Context Menu")).to_be_visible()

def test_FR_CD_01_Challenging_DOM(page: Page):
    """FR-CD-01: The Challenging DOM page must display a table with specific heade
    page.goto(f"{BASE_URL}/challenging_dom")
    expect(page.get_by_role("heading", name="Challenging DOM")).to_be_visible()
    table_headers = page.locator("table th").all_text_contents()
    expected_headers = ["Lorem", "Ipsum", "Dolor", "Sit", "Amet", "Diceret", "Acti
    assert all(header in table_headers for header in expected_headers)
    expect(page.locator("table tbody tr")).to_have_count(10) # Default 10 rows
    expect(page.locator("table tbody tr").first.locator("td a", has_text="edit")).

```

```

expect(page.locator("table tbody tr").first.locator("td a", has_text="delete"))

def test_FR_CD_02_Challenging_DOM(page: Page):
    """FR-CD-02: The Challenging DOM page must describe that locators are intention
    page.goto(f"{BASE_URL}/challenging_dom")
    expect(page.locator(".large-12.columns")).to_contain_text("The DOM on this pag
    expect(page.locator(".large-12.columns")).to_contain_text("Unique IDs are not
    expect(page.locator(".large-12.columns")).to_contain_text("A canvas element is

def test_FR_EI_01_Exit_Intent(page: Page):
    """FR-EI-01: A modal window must automatically display when the mouse moves ou
    page.goto(f"{BASE_URL}/exit_intent")
    modal = page.locator(".modal")
    expect(modal).not_to_be_visible() # Initially not visible

    # Move mouse out of viewport to trigger modal
    page.mouse.move(100, 100) # Move to a point inside
    page.mouse.move(100, -1) # Move outside the top edge
    expect(modal).to_be_visible()
    expect(modal.get_by_role("heading", name="THIS IS A MODAL WINDOW")).to_be_visi

def test_FR_EI_02_Exit_Intent(page: Page):
    """FR-EI-02: The exit intent modal must include a 'Close' control that dismiss
    page.goto(f"{BASE_URL}/exit_intent")
    modal = page.locator(".modal")

    # Trigger modal
    page.mouse.move(100, 100)
    page.mouse.move(100, -1)
    expect(modal).to_be_visible()

    # Close modal
    modal.get_by_text("Close").click()
    expect(modal).not_to_be_visible()

def test_FR_JQM_01_JQuery_UI_Menu(page: Page):
    """FR-JQM-01: The JQuery UI Menu page must display a JQuery UI menu demonstrat
    page.goto(f"{BASE_URL}/jqueryui/menu")
    expect(page.get_by_role("heading", name="JQuery UI - Menu")).to_be_visible()
    menu = page.locator("#menu")
    expect(menu).to_be_visible()
    expect(menu.get_by_role("link", name="Enabled")).to_be_visible()
    expect(menu.get_by_role("link", name="Back to JQuery UI")).not_to_be_visible()

```

```

# Hover to reveal nested items
menu.get_by_role("link", name="Enabled").hover()
expect(menu.get_by_role("link", name="Downloads")).to_be_visible()
menu.get_by_role("link", name="Downloads").hover()
expect(menu.get_by_role("link", name="PDF")).to_be_visible()

def test_FR_JQM_02_JQuery_UI_Menu(page: Page):
    """FR-JQM-02: The JQuery UI Menu page must explain that element visibility is
    page.goto(f"{BASE_URL}/jqueryui/menu")
    expect(page.locator(".example")).to_contain_text("This example demonstrates th
    expect(page.locator(".example")).to_contain_text("The visibility of the sub-me

def test_FR_JE_01_JavaScript_Error(page: Page):
    """FR-JE-01: The JavaScript Error page must contain a JavaScript error trigger
    # Listen for console errors
    errors = []
    page.on("console", lambda msg: errors.append(msg.text) if msg.type == "error"
    page.goto(f"{BASE_URL}/javascript_error")
    # Wait for potential errors to be logged
    page.wait_for_load_state("networkidle")
    assert any("Cannot read properties of undefined (reading 'xyz')" in error for
               "Expected JavaScript error 'Cannot read properties of undefined (reading '

def test_FR_JE_02_JavaScript_Error(page: Page):
    """FR-JE-02: The JavaScript Error page must describe that this can be a proble
    page.goto(f"{BASE_URL}/javascript_error")
    expect(page.get_by_role("heading", name="JavaScript error")).to_be_visible()
    expect(page.locator(".example")).to_contain_text("This page has a JavaScript e

def test_FR_LD_01_Large_Deep_DOM(page: Page):
    """FR-LD-01: The Large & Deep DOM page must present a deeply nested DOM struct
    page.goto(f"{BASE_URL}/large")
    expect(page.get_by_role("heading", name="Large & Deep DOM")).to_be_visible()
    expect(page.get_by_text("No Siblings")).to_be_visible()
    expect(page.get_by_text("Siblings")).to_be_visible()
    # Check for a specific cell to confirm tabular layout
    expect(page.locator("#sibling-1\\.1")).to_be_visible()
    expect(page.locator("#sibling-50\\.50")).to_be_visible()

def test_FR_LD_02_Large_Deep_DOM(page: Page):
    """FR-LD-02: The Large & Deep DOM page must describe that this layout demonstr
    page.goto(f"{BASE_URL}/large")

```



```

expect(page.locator(".example")).to_contain_text("This page is designed to dem

def test_FR_IS_01_Infinite_Scroll(page: Page):
    """FR-IS-01: The Infinite Scroll page must implement infinite scrolling such t
    page.goto(f"{BASE_URL}/infinite_scroll")
    expect(page.get_by_role("heading", name="Infinite Scroll")).to_be_visible()
    initial_div_count = page.locator(".jscroll-added").count()

    # Scroll down to trigger more content
    page.mouse.wheel(0, 10000) # Scroll a large amount
    page.wait_for_timeout(2000) # Give time for content to load

    new_div_count = page.locator(".jscroll-added").count()
    assert new_div_count > initial_div_count, "New content blocks should load on s

def test_FR_IS_02_Infinite_Scroll(page: Page):
    """FR-IS-02: The Infinite Scroll page must contain a link or text 'next page'.
    page.goto(f"{BASE_URL}/infinite_scroll")
    expect(page.get_by_text("next page")).to_be_visible()

def test_FR_DL_01_File_Download(page: Page):
    """FR-DL-01: The File Download page must present a list of files as links unde
    page.goto(f"{BASE_URL}/download")
    expect(page.get_by_role("heading", name="File Downloader")).to_be_visible()
    expect(page.locator(".example a")).to_have_count(10) # Assuming there are 10 f
    expect(page.locator(".example a").first).to_be_visible()

def test_FR_DL_02_File_Download(page: Page):
    """FR-DL-02: Clicking on any file name link must initiate a download of the co
    page.goto(f"{BASE_URL}/download")
    download_link = page.locator(".example a").first

    with page.expect_download() as download_info:
        download_link.click()

    download = download_info.value
    expect(download.suggested_filename).not_to_be_empty()
    # You can save the file and check its content if needed
    # download.save_as(os.path.join(tmp_path, download.suggested_filename))

def test_FR_FP_01_Forgot_Password(page: Page):
    """FR-FP-01: The Forgot Password page must display a form with an 'E-mail' fie
    page.goto(f"{BASE_URL}/forgot_password")

```



```
expect(page.get_by_role("heading", name="Forgot Password")).to_be_visible()
expect(page.get_by_label("E-mail")).to_be_visible()
expect(page.get_by_role("button", name="Retrieve password")).to_be_visible()

def test_FR_FP_02_Forgot_Password(page: Page):
    """FR-FP-02: Submitting the Forgot Password form must attempt to retrieve a pa
    page.goto(f"{BASE_URL}/forgot_password")
    page.get_by_label("E-mail").fill("test@example.com")
    page.get_by_role("button", name="Retrieve password").click()
    expect(page.get_by_role("heading", name="Internal Server Error")).to_be_visibl

def test_FR_GL_01_Geolocation(page: Page):
    """FR-GL-01: The Geolocation page must display heading 'Geolocation' and a but
    page.goto(f"{BASE_URL}/geolocation")
    expect(page.get_by_role("heading", name="Geolocation")).to_be_visible()
    expect(page.get_by_role("button", name="Where am I?")).to_be_visible()

def test_FR_GL_02_Geolocation(page: Page):
    """FR-GL-02: Clicking 'Where am I?' must attempt to access and display the use
    page.goto(f"{BASE_URL}/geolocation")

    # Mock geolocation to provide a consistent result
    page.context.set_geolocation({"latitude": 34.052235, "longitude": -118.243683})

    page.get_by_role("button", name="Where am I?").click()

    # Expect the coordinates to be displayed
    expect(page.locator("#lat-value")).to_have_text("34.052235")
    expect(page.locator("#long-value")).to_have_text("-118.243683")

def test_FR_FM_01_Floating_Menu(page: Page):
    """FR-FM-01: The Floating Menu page must show a floating menu that remains vis
    page.goto(f"{BASE_URL}/floating_menu")
    menu = page.locator("#menu")
    expect(menu).to_be_visible()

    initial_y = menu.bounding_box()["y"]
    page.mouse.wheel(0, 500) # Scroll down
    page.wait_for_timeout(500) # Give time for scroll and potential re-positioning
    current_y = menu.bounding_box()["y"]

    # The menu should stay in roughly the same viewport position (fixed)
    # The exact Y coordinate might change slightly due to browser rendering,
```

```

# but it should not scroll out of view.
# A more robust check would be to check its position relative to the viewport.
# For now, we check it's still visible and its Y position is within a reasonable
expect(menu).to_be_visible()
assert abs(current_y - initial_y) < 50 # Should not move significantly in Y axis

def test_FR_FM_02_Floating_Menu(page: Page):
    """FR-FM-02: The Floating Menu page's main content must consist of multiple paragraphs"""
    page.goto(f"{BASE_URL}/floating_menu")
    expect(page.locator("#content p")).to_have_count(20) # Assuming 20 paragraphs
    expect(page.locator("#content p").first).to_contain_text("Lorem ipsum dolor si")

def test_FR_SD_01_Shadow_DOM(page: Page):
    """FR-SD-01: The Shadow DOM page must demonstrate shadow DOM usage, including"""
    page.goto(f"{BASE_URL}/shadowdom")
    expect(page.get_by_role("heading", name="Shadow DOM")).to_be_visible()
    expect(page.locator("h4")).to_have_text("My default text")

    # Content inside shadow DOM is not directly accessible by standard locators
    # We can access it via JavaScript evaluation or by traversing shadow roots
    shadow_host = page.locator("my-paragraph").first
    shadow_root_content = shadow_host.evaluate("node => node.shadowRoot.querySelector('p')")
    assert "Let's have some different text! In a list!" in shadow_root_content

def test_FR_SD_02_Shadow_DOM(page: Page):
    """FR-SD-02: Some content on the Shadow DOM page must be enclosed in shadow DOM"""
    page.goto(f"{BASE_URL}/shadowdom")

    # Attempt to query for content that is known to be inside shadow DOM using a regular expression
    # This should return null or an empty list, demonstrating it's not in the light DOM
    # For example, the <span> inside my-paragraph's shadow root.

    # First, verify the shadow host is present in the light DOM
    expect(page.locator("my-paragraph")).to_be_visible()

    # Now, use page.evaluate() to run document.querySelector('span') in the light DOM
    # and assert it returns null for the shadow DOM content.
    # The specific span we're looking for is inside the shadow root of 'my-paragraph'
    # A direct query for 'span' in the light DOM should not find it.
    span_in_light_dom = page.evaluate("document.querySelector('span')")
    assert span_in_light_dom is None, "A span element from shadow DOM was unexpectedly found in the light DOM"

    # To further demonstrate, we can try to find the 'li' element which is also in

```

```

li_in_light_dom = page.evaluate("document.querySelector('li')")
assert li_in_light_dom is None, "An li element from shadow DOM was unexpectedl

def test_FR_FR_01_Frames(page: Page):
    """FR-FR-01: The Frames overview page must display a list of frame-related exa
    page.goto(f"{BASE_URL}/frames")
    expect(page.get_by_role("heading", name="Frames")).to_be_visible()
    expect(page.get_by_role("link", name="Nested Frames")).to_be_visible()
    expect(page.get_by_role("link", name="iFrame")).to_be_visible()

def test_FR_FR_02_Frames(page: Page):
    """FR-FR-02: Selecting 'Nested Frames' must navigate to a page with multiple e
    page.goto(f"{BASE_URL}/frames")
    page.get_by_role("link", name="Nested Frames").click()
    expect(page).to_have_url(f"{BASE_URL}/nested_frames")
    expect(page.frame_locator("frame[name='frame-top']")).to_be_visible()
    expect(page.frame_locator("frame[name='frame-left']")).to_be_visible()
    expect(page.frame_locator("frame[name='frame-right']")).to_be_visible()
    expect(page.frame_locator("frame[name='frame-bottom']")).to_be_visible()

def test_FR_FR_03_Frames(page: Page):
    """FR-FR-03: Selecting 'iFrame' must navigate to a page with an editable rich
    page.goto(f"{BASE_URL}/frames")
    page.get_by_role("link", name="iFrame").click()
    expect(page).to_have_url(f"{BASE_URL}/iframe")
    expect(page.get_by_role("heading", name="An iFrame containing the TinyMCE WYSI
    expect(page.frame_locator("#mce_0_ifr")).to_be_visible()

def test_FR_WIN_01_Windows(page: Page):
    """FR-WIN-01: Clicking 'Click Here' on the Windows page must open a new browse
    page.goto(f"{BASE_URL}/windows")

    with page.context.expect_page() as new_page_info:
        page.get_by_role("link", name="Click Here").click()

    new_page = new_page_info.value
    expect(new_page).not_to_be_null()
    expect(new_page).to_have_url(f"{BASE_URL}/windows/new")
    expect(new_page.get_by_role("heading", name="New Window")).to_be_visible()
    new_page.close()

def test_FR_WIN_02_Windows(page: Page):
    """FR-WIN-02: The multiple_windows page must provide a multi-window demonstrat

```

```

page.goto(f"{BASE_URL}/multiple_windows")
expect(page.get_by_role("heading", name="Opening a new window")).to_be_visible()
expect(page.get_by_role("link", name="Click Here")).to_be_visible()

def test_FR_SHC_01_Shifting_Content(page: Page):
    """FR-SHC-01: The Shifting Content page must list three examples describing el
    page.goto(f"{BASE_URL}/shifting_content")
    expect(page.get_by_role("heading", name="Shifting Content")).to_be_visible()
    expect(page.locator(".example")).to_contain_text("This example demonstrates ho
    expect(page.get_by_role("link", name="Example 1: Menu Element")).to_be_visible
    expect(page.get_by_role("link", name="Example 2: An image")).to_be_visible()
    expect(page.get_by_role("link", name="Example 3: List")).to_be_visible()

def test_FR_SHC_02_Shifting_Content(page: Page):
    """FR-SHC-02: On reload, elements' positions on the Shifting Content page must
    page.goto(f"{BASE_URL}/shifting_content/menu") # Navigate to a specific shifti
    menu_element = page.locator("#menu")

    initial_x = menu_element.bounding_box()["x"]
    page.reload()
    page.wait_for_load_state("domcontentloaded")
    current_x = menu_element.bounding_box()["x"]

    # Expect a slight shift, so the positions should not be exactly the same
    assert initial_x != current_x, "Menu element position should shift on reload."
    # And the shift should be small, not a complete relocation
    assert abs(initial_x - current_x) < 100, "Menu element shifted too much, indic

def test_FR_MISC_01_Miscellaneous_Examples(page: Page):
    """FR-MISC-01: Other examples listed on the home page (e.g., 'Slow Resources',
    # Test Slow Resources
    page.goto(f"{BASE_URL}/slow_resources")
    expect(page.get_by_role("heading", name="Slow Resources")).to_be_visible()
    expect(page.locator(".example")).to_contain_text("This page is designed to dem

    # Test Secure File Download (requires login, so just check page title/heading)
    page.goto(f"{BASE_URL}/download_secure")
    expect(page.get_by_role("heading", name="Basic Auth")).to_be_visible() # It re

def test_NFR_U_01_Usability_Discoverability(page: Page):
    """NFR-U-01: Page structures must be simple and minimalistic, making intended
    page.goto(f"{BASE_URL}/checkboxes")
    expect(page.get_by_role("heading", name="Checkboxes")).to_be_visible()

```

```

# Assert that checkboxes are clearly visible and clickable
expect(page.locator("input[type='checkbox']").first).to_be_visible()
expect(page.locator("input[type='checkbox']").first).to_be_enabled()

page.goto(f"{BASE_URL}/login")
expect(page.get_by_role("heading", name="Login Page")).to_be_visible()
# Assert that input fields and login button are clearly visible and interactive
expect(page.get_by_label("Username")).to_be_visible()
expect(page.get_by_label("Password")).to_be_visible()
expect(page.get_by_role("button", name="Login")).to_be_visible()

def test_NFR_U_02_Usability_Explanatory_Text(page: Page):
    """NFR-U-02: Each example page must include explanatory text near the top desc
    page.goto(f"{BASE_URL}/typos")
    expect(page.get_by_role("heading", name="Typos")).to_be_visible()
    expect(page.locator(".example p").first).to_contain_text("This example demonstr

    page.goto(f"{BASE_URL}/dynamic_content")
    expect(page.get_by_role("heading", name="Dynamic Content")).to_be_visible()
    expect(page.locator(".example")).to_contain_text("This example demonstrates ch

def test_NFR_U_03_Usability_Control_Clarity(page: Page):
    """NFR-U-03: Important interactive controls must be visually distinguishable a
    page.goto(f"{BASE_URL}/login")
    login_button = page.get_by_role("button", name="Login")
    expect(login_button).to_be_visible()
    expect(login_button).to_have_css("background-color", "rgb(42, 106, 167)") # Ex
    expect(login_button).to_have_text("Login")

    page.goto(f"{BASE_URL}/add_remove_elements/")
    add_element_button = page.get_by_role("button", name="Add Element")
    expect(add_element_button).to_be_visible()
    expect(add_element_button).to_have_text("Add Element")

def test_NFR_R_01_Reliability_Page_Accessibility(page: Page):
    """NFR-R-01: All example pages listed on the home page must be accessible via
    page.goto(BASE_URL)
    links = page.locator("#content ul li a").all()

    # List of pages known to have intentional non-200 responses or special handlin
    # /status_codes/* are handled by specific tests
    # /download_secure requires basic auth
    # /javascript_error has an intentional JS error, but page loads

```

```

# /forgot_password POSTs to a 500
# /login POSTs to /secure or /login with error
# /dynamic_loading/1 and /dynamic_loading/2 are sub-pages

# Filter out links that are not direct page loads or are handled elsewhere
excluded_paths = ["/status_codes", "/download_secure", "/javascript_error", "/"

for link in links:
    href = link.get_attribute("href")
    if href and not any(href.startswith(f"{BASE_URL}{ep}") for ep in excluded_
        with page.expect_response(lambda response: response.status == 200) as
            link.click()
        expect(page).to_have_url(href) # Ensure navigation
        expect(response_info.value.status).to_equal(200)
        page.go_back() # Return to home page for next link

def test_NFR_R_02_Reliability_Randomized_Behavior_Stability(page: Page):
    """NFR-R-02: Randomized behaviors must not cause JavaScript errors that preven
    # Test /typos
    errors = []
    handler = lambda msg: errors.append(msg.text) if msg.type == "error" else None
    page.on("console", handler)

    for _ in range(5): # Refresh multiple times
        page.goto(f"{BASE_URL}/typos")
        page.wait_for_load_state("networkidle")
        expect(page.get_by_role("heading", name="Typos")).to_be_visible()
        assert not errors, f"JavaScript errors found on /typos: {errors}"
        errors.clear() # Clear errors for next iteration

    # Test /dynamic_content
    for _ in range(5):
        page.goto(f"{BASE_URL}/dynamic_content")
        page.wait_for_load_state("networkidle")
        expect(page.get_by_role("heading", name="Dynamic Content")).to_be_visible(
        assert not errors, f"JavaScript errors found on /dynamic_content: {errors}"
        errors.clear()

    # Test /notification_message_rendered
    for _ in range(5):
        page.goto(f"{BASE_URL}/notification_message_rendered")
        page.wait_for_load_state("networkidle")
        expect(page.get_by_role("heading", name="Notification Message")).to_be_vis

```

```

expect(page.locator("#flash")).to_be_visible()
page.get_by_role("link", name="Click here").click()
page.wait_for_load_state("networkidle")
expect(page.locator("#flash")).to_be_visible()
assert not errors, f"JavaScript errors found on /notification_message_rend
errors.clear()

```

```

page.remove_listener("console", handler) # Remove the listener

```

```

def test_EDGE_TYPO_01_Typos(page: Page):
    """EDGE-TYPO-01: Due to randomness, a typo on the /typos page is not guarantee
    page.goto(f"{BASE_URL}/typos")
    expect(page.get_by_role("heading", name="Typos")).to_be_visible()
    paragraph = page.locator(".example p").nth(1)

    # Acknowledging limitation: To fully test the 'typo may not be present' aspect
    # the test should ideally reload the page multiple times and assert that the t
    # is sometimes present and sometimes absent.
    # For a single test run, we can only check the current state.
    # We assert that the page loads successfully and contains the base text.
    expect(paragraph).to_contain_text("Sometimes you and I")
    # We cannot assert the absence of a typo, only that the page loads.
    # The purpose of this edge case is to highlight the randomness, not to fail if

def test_NEG_TYPO_01_Typos(page: Page):
    """NEG-TYPO-01: Automation relying on exact text match on the /typos page may
    page.goto(f"{BASE_URL}/typos")
    paragraph = page.locator(".example p").nth(1)

    # To demonstrate automation fragility, assert for only the correct text.
    # This test is expected to intermittently fail due to the intentional randomne
    # thus proving the negative requirement.
    # We use pytest.raises to catch the expected assertion failure.

    # Get the actual text content
    actual_text = paragraph.text_content().strip()

    # Define the expected correct text
    expected_correct_text = "Sometimes you and I get a typo."

    # Assert that the text is only the correct text.
    # This assertion will fail if the typo "won,t" is present.
    # We wrap this in a try-except block to show it's an expected intermittent fai

```



```

try:
    expect(paragraph).to_have_text(expected_correct_text)
    # If it passes, it means the typo was NOT present on this run.
    pytest.fail(f"Test passed unexpectedly. Typo was not present. Actual: '{actual_text}'")
except AssertionError:
    # If it fails, it means the typo WAS present, which is the intended demonstration
    assert "won't get a typo." in actual_text, \
        f"Assertion failed, but not due to expected typo. Actual: '{actual_text}'"
    print(f"\nNEG-TYP0-01: Test failed as expected due to typo. Actual text: '{actual_text}'")
    # This test is designed to intermittently fail, so we don't re-raise.
    # In a real CI, this would be marked as flaky or an expected failure.

def test_EDGE_DCNT_01_Dynamic_Content(page: Page):
    """EDGE-DCNT-01: Text and images on the /dynamic_content page may change on every reload"""
    page.goto(f"{BASE_URL}/dynamic_content")
    initial_content = page.locator("#content").inner_html()
    page.reload()
    # Use assert with inner_html for correct comparison
    assert page.locator("#content").inner_html() != initial_content, "Content should have changed"

def test_EDGE_NM_01_Notification_Messages(page: Page):
    """EDGE-NM-01: Different notification messages can appear after each 'Click here'"""
    page.goto(f"{BASE_URL}/notification_message_rendered")

    messages = set()
    for _ in range(3): # Click multiple times to observe different messages
        flash_message = page.locator("#flash")
        expect(flash_message).to_be_visible()
        messages.add(flash_message.text_content().strip())

    page.get_by_role("link", name="Click here").click()
    page.wait_for_load_state("networkidle")

    # Assert that at least two different messages were observed, indicating random
    assert len(messages) > 1, f"Expected different messages, but only found: {messages}"

def test_EDGE_DE_01_Disappearing_Elements(page: Page):
    """EDGE-DE-01: Certain menu items on the /disappearing_elements page may be absent"""
    page.goto(f"{BASE_URL}/disappearing_elements")
    expect(page.get_by_role("heading", name="Disappearing Elements")).to_be_visible()

    # Acknowledging limitation: To fully test the 'may be absent' aspect,
    # the test should ideally reload the page multiple times and assert that the '

```



```

# is sometimes present and sometimes absent.
# For a single test, we can only check its state on one load.
# We assert that the page loads successfully. The absence of 'Gallery' is a va
expect(page.get_by_role("link", name="Home")).to_be_visible()

# We can check if 'Gallery' is present on this specific load.
# The test passes if the page loads without error, regardless of Gallery's pre
gallery_link = page.get_by_role("link", name="Gallery")
if gallery_link.count() > 0:
    expect(gallery_link).to_be_visible()
else:
    # This branch is also a valid outcome for the edge case.
    pass

def test_NEG_FA_01_Form_Authentication_Login(page: Page):
    """NEG-FA-01: Entering wrong username, wrong password, or both on the /login p
    page.goto(f"{BASE_URL}/login")
    page.get_by_label("Username").fill("wronguser")
    page.get_by_label("Password").fill("wrongpass")
    page.get_by_role("button", name="Login").click()
    expect(page).to_have_url(f"{BASE_URL}/login")
    expect(page.locator("#flash.error")).to_be_visible()
    expect(page.locator("#flash.error")).to_contain_text("Your username is invalid

def test_EDGE_FA_01_Form_Authentication_Login(page: Page):
    """EDGE-FA-01: Submitting empty username or password fields on the /login page
    page.goto(f"{BASE_URL}/login")

    # Test empty username
    page.get_by_label("Username").fill("")
    page.get_by_label("Password").fill("SuperSecretPassword!")
    page.get_by_role("button", name="Login").click()
    expect(page.locator("#flash.error")).to_be_visible()
    expect(page.locator("#flash.error")).to_contain_text("Your username is invalid
    page.reload() # Reset page

    # Test empty password
    page.get_by_label("Username").fill("tomsmith")
    page.get_by_label("Password").fill("")
    page.get_by_role("button", name="Login").click()
    expect(page.locator("#flash.error")).to_be_visible()
    expect(page.locator("#flash.error")).to_contain_text("Your username is invalid
    page.reload() # Reset page

```

```

# Test both empty
page.get_by_label("Username").fill("")
page.get_by_label("Password").fill("")
page.get_by_role("button", name="Login").click()
expect(page.locator("#flash.error")).to_be_visible()
expect(page.locator("#flash.error")).to_contain_text("Your username is invalid")

def test_EDGE_FP_01_Forgot_Password(page: Page):
    """EDGE-FP-01: Submitting the /forgot_password form with a blank or invalid email"""
    page.goto(f"{BASE_URL}/forgot_password")

    # Test blank email
    page.get_by_label("E-mail").fill("")
    page.get_by_role("button", name="Retrieve password").click()
    # The site navigates to /email_sent with an Internal Server Error, which is it
    expect(page).to_have_url(f"{BASE_URL}/email_sent")
    expect(page.get_by_role("heading", name="Internal Server Error")).to_be_visible()
    page.go_back()

    # Test invalid email format (the browser's default validation might prevent submission
    # but if it allows, the server-side behavior is expected to be similar to blank email
    # The site's input type is 'text', so browser validation for email format is not applicable
    page.get_by_label("E-mail").fill("invalid-email")
    page.get_by_role("button", name="Retrieve password").click()
    expect(page).to_have_url(f"{BASE_URL}/email_sent")
    expect(page.get_by_role("heading", name="Internal Server Error")).to_be_visible()

def test_EDGE_IN_01_Inputs(page: Page):
    """EDGE-IN-01: Entering non-numeric characters into the numeric field on the /inputs page"""
    page.goto(f"{BASE_URL}/inputs")
    input_field = page.locator("input[type='number']")

    input_field.fill("abc")
    # Browser default behavior for type="number" is to ignore non-numeric input
    expect(input_field).to_have_value("") # Value should remain empty or be coerced to 0

    input_field.fill("123abc456")
    expect(input_field).to_have_value("123456") # Only numeric parts are kept

    # Ensure page remains functional
    expect(page.get_by_role("heading", name="Inputs")).to_be_visible()

```

```

def test_NEG_UP_01_File_Upload(page: Page):
    """NEG-UP-01: Attempting to submit the /upload form without choosing a file sh
page.goto(f"{BASE_URL}/upload")

    # The file input is required, so the browser will prevent submission.
    # Playwright's click() on a submit button for a form with a required file input
    # will not trigger a navigation if the browser prevents it.
    # We can assert that the page remains on the same URL and no success message appears.

    # Click the upload button without selecting a file
page.locator("#file-submit").click()

    # The page should not navigate away, and no "File Uploaded!" message should appear.
page.expect(page).to_have_url(f"{BASE_URL}/upload")
expect(page.get_by_role("heading", name="File Uploaded!")).not_to_be_visible()
    # The browser's default validation message is not directly accessible by Playwright
    # but the lack of navigation and success message confirms the browser handled the error.

def test_EDGE_UP_01_File_Upload(page: Page, tmp_path):
    """EDGE-UP-01: Uploading large files or unsupported formats on the /upload page
page.goto(f"{BASE_URL}/upload")

    # Create a large dummy file (e.g., 10MB)
large_file_name = "large_dummy.txt"
large_file_path = tmp_path / large_file_name
    # Create a file with 10MB of 'a' characters
with open(large_file_path, "wb") as f:
    f.write(b"a" * 10 * 1024 * 1024) # 10 MB

page.locator("#file-upload").set_input_files(str(large_file_path))
page.locator("#file-submit").click()

    # The server might return an error or take a long time.
    # We expect the page not to crash and to display some result (success or error)
    # The-internet.herokuapp.com typically shows "Internal Server Error" for large uploads.
page.expect(page).to_have_text("Internal Server Error") # Expect some content to be visible
    # Assert that it's not a blank page or a browser crash.
    # The specific outcome might be "File Uploaded!" or "Internal Server Error".
    # We'll check for either.
try:
    expect(page.get_by_role("heading", name="File Uploaded!")).to_be_visible()
except Exception:
    expect(page.get_by_role("heading", name="Internal Server Error")).to_be_visible()

```

```

def test_NEG_JA_01_JavaScript_Alerts(page: Page):
    """NEG-JA-01: Canceling a confirm dialog or dismissing a prompt without entry
    page.goto(f"{BASE_URL}/javascript_alerts")

    # Test JS Confirm - Cancel
    page.on("dialog", lambda dialog: dialog.dismiss())
    page.get_by_role("button", name="Click for JS Confirm").click()
    expect(page.locator("#result")).to_have_text("You clicked: Cancel")

    # Test JS Prompt - Cancel
    page.on("dialog", lambda dialog: dialog.dismiss())
    page.get_by_role("button", name="Click for JS Prompt").click()
    expect(page.locator("#result")).to_have_text("You entered: null")

    # Ensure no JS errors occurred
    errors = []
    page.on("console", lambda msg: errors.append(msg.text) if msg.type == "error")
    page.wait_for_load_state("networkidle")
    assert not errors, f"JavaScript errors found: {errors}"

def test_EDGE_JA_01_JavaScript_Alerts(page: Page):
    """EDGE-JA-01: Multiple rapid clicks on alert buttons on the /javascript_alert
    page.goto(f"{BASE_URL}/javascript_alerts")

    # Rapidly click the alert button multiple times
    for _ in range(5):
        # Each click will open a dialog, which we immediately accept
        page.on("dialog", lambda dialog: dialog.accept())
        page.get_by_role("button", name="Click for JS Alert").click()
        expect(page.locator("#result")).to_have_text("You successfully clicked an
        # Small wait to ensure dialog is processed before next click, but still ra
        page.wait_for_timeout(100)

    # Ensure the page is still functional after rapid interactions
    expect(page.get_by_role("heading", name="JavaScript Alerts")).to_be_visible()

def test_EDGE_CM_01_Context_Menu(page: Page):
    """EDGE-CM-01: Right-clicking outside the specified box on the /context_menu p
    page.goto(f"{BASE_URL}/context_menu")

    # Right-click on the body, outside the #hot-spot
    page.locator("body").click(button="right", position={"x": 10, "y": 10})

```

```

# Since Playwright cannot directly inspect the native browser context menu,
# we assert that the custom alert (which would be triggered by the custom menu
# is NOT triggered. This implies the default browser menu appeared.

# Set up a dialog listener that will fail the test if a dialog appears
def on_dialog(dialog):
    pytest.fail(f"Unexpected dialog appeared: {dialog.message}")

page.on("dialog", on_dialog)

# Wait a short period to ensure no dialog appears
page.wait_for_timeout(1000) # Give time for any potential dialog to appear

# If no dialog appeared, the test passes, implying default context menu.
expect(page.get_by_role("heading", name="Context Menu")).to_be_visible()
page.remove_listener("dialog", on_dialog) # Clean up listener

def test_NEG_CM_01_Context_Menu(page: Page):
    """NEG-CM-01: If the user ignores the alert triggered by context menu selectio
    page.goto(f"{BASE_URL}/context_menu")
    hot_spot = page.locator("#hot-spot")

    # Trigger the alert
    dialog_triggered = False
    def on_dialog(dialog):
        nonlocal dialog_triggered
        dialog_triggered = True
        # Do not accept/dismiss immediately, simulate user ignoring
        # Playwright will automatically dismiss after the test if not handled,
        # but we can explicitly dismiss after a short wait.
        page.wait_for_timeout(2000) # Simulate user ignoring for 2 seconds
        dialog.accept()

    page.on("dialog", on_dialog)
    hot_spot.click(button="right")

    # Wait for the dialog to be triggered and handled
    page.wait_for_function("() => window.dialog_triggered", timeout=5000) # Custom

    # After dialog is dismissed, ensure page is still functional
    expect(page.get_by_role("heading", name="Context Menu")).to_be_visible()
    expect(hot_spot).to_be_visible()

```

```
page.remove_listener("dialog", on_dialog)

def test_EDGE_HV_01_Hovers(page: Page):
    """EDGE-HV-01: Hover events on /hovers page may not behave as expected in mobile"""
    page.goto(f"{BASE_URL}/hovers")

    # Simulate a touch event (tap) instead of hover
    # Playwright's `tap()` method simulates a touch tap.
    figure = page.locator(".figure").first
    figcaption = figure.locator(".figcaption")

    expect(figcaption).not_to_be_visible()

    # Tap on the figure
    figure.tap()

    # In a touch environment, a tap often reveals hover-like content.
    # The site's JS might be designed to show the figcaption on click/tap as well.
    expect(figcaption).to_be_visible()
    expect(figcaption.get_by_text("name: user1")).to_be_visible()

    # Tap again to dismiss (common touch behavior) or tap outside
    page.locator("body").tap()
    expect(figcaption).not_to_be_visible()

def test_EDGE_HS_01_Horizontal_Slider(page: Page):
    """EDGE-HS-01: The /horizontal_slider should support both keyboard and mouse input"""
    page.goto(f"{BASE_URL}/horizontal_slider")
    slider = page.locator("input[type='range']")
    value_indicator = page.locator("#range")

    # Test mouse drag to max
    slider.drag_to(slider, target_position={"x": slider.bounding_box()["x"] + slider.bounding_box()["width"]})
    expect(value_indicator).to_have_text("5")

    # Test keyboard beyond max
    slider.press("ArrowRight")
    expect(value_indicator).to_have_text("5") # Should clamp at 5

    # Test mouse drag to min
    slider.drag_to(slider, target_position={"x": slider.bounding_box()["x"], "y": slider.bounding_box()["y"]})
    expect(value_indicator).to_have_text("0")
```

```
# Test keyboard beyond min
slider.press("ArrowLeft")
expect(value_indicator).to_have_text("0") # Should clamp at 0

def test_EDGE_LD_01_Large_Deep_DOM(page: Page):
    """EDGE-LD-01: The /large page, despite its large DOM, should remain responsive"""
    errors = []
    page.on("console", lambda msg: errors.append(msg.text) if msg.type == "error")

    page.goto(f"{BASE_URL}/large")
    expect(page.get_by_role("heading", name="Large & Deep DOM")).to_be_visible()

    # Scroll rapidly
    for _ in range(5):
        page.mouse.wheel(0, 5000)
        page.wait_for_timeout(100)
        page.mouse.wheel(0, -5000)
        page.wait_for_timeout(100)

    # Check for responsiveness (e.g., a known element is still interactable)
    expect(page.locator("#sibling-1\\.1")).to_be_visible()

    # Assert no critical JavaScript errors
    assert not errors, f"JavaScript errors found on /large page: {errors}"

def test_NEG_IS_01_Infinite_Scroll(page: Page):
    """NEG-IS-01: Scrolling beyond available dynamically loaded content on the /infinite_scroll page"""
    errors = []
    page.on("console", lambda msg: errors.append(msg.text) if msg.type == "error")

    page.goto(f"{BASE_URL}/infinite_scroll")
    expect(page.get_by_role("heading", name="Infinite Scroll")).to_be_visible()

    initial_div_count = page.locator(".jscroll-added").count()

    # Scroll repeatedly to the bottom
    for _ in range(10): # Scroll many times
        page.mouse.wheel(0, 10000) # Scroll a large amount
        page.wait_for_timeout(500) # Give time for content to load

    final_div_count = page.locator(".jscroll-added").count()

    # Assert that content loading eventually stops (count doesn't increase indefinitely)
```

```

# This is hard to assert definitively without knowing the max content.
# A better check is that no errors occur and the page remains stable.

# Scroll one more time and check if count changed. It should not.
page.mouse.wheel(0, 10000)
page.wait_for_timeout(500)
final_div_count_after_extra_scroll = page.locator(".jscroll-added").count()

assert final_div_count_after_extra_scroll == final_div_count, \
    "Content continued to load after reaching the end of available content."

assert not errors, f"JavaScript errors found during infinite scroll: {errors}"

def test_EDGE_FM_01_Floating_Menu(page: Page):
    """EDGE-FM-01: Very fast scrolling on the /floating_menu page should not detach
    page.goto(f"{BASE_URL}/floating_menu")
    menu = page.locator("#menu")
    expect(menu).to_be_visible()

    # Get initial position relative to viewport
    initial_bounding_box = menu.bounding_box()
    initial_viewport_y = initial_bounding_box["y"]

    # Scroll very rapidly up and down
    for _ in range(10):
        page.mouse.wheel(0, 5000) # Scroll down
        page.wait_for_timeout(50) # Very short wait
        page.mouse.wheel(0, -5000) # Scroll up
        page.wait_for_timeout(50)

    # After rapid scrolling, check if the menu is still visible and in roughly the
    expect(menu).to_be_visible()
    final_bounding_box = menu.bounding_box()
    final_viewport_y = final_bounding_box["y"]

    # Allow for minor rendering differences, but it should be largely stable
    assert abs(final_viewport_y - initial_viewport_y) < 20, \
        f"Floating menu position shifted significantly after rapid scrolling. Init

def test_EDGE_SHC_01_Shifting_Content(page: Page):
    """EDGE-SHC-01: Pixel-level shifts on each load of the /shifting_content page
    page.goto(f"{BASE_URL}/shifting_content/menu") # Navigate to a specific shifti
    menu_element = page.locator("#menu")

```



```

initial_x_positions = []

for _ in range(3): # Reload multiple times to observe shifts
    page.reload()
    page.wait_for_load_state("domcontentloaded")
    initial_x_positions.append(menu_element.bounding_box()["x"])

# Assert that not all positions are identical, demonstrating the shift
assert len(set(initial_x_positions)) > 1, \
    f"Menu element position did not shift across reloads. Positions: {initial_

# This test passes by demonstrating the shift, which would break fixed-positio

def test_NEG_GL_01_Geolocation(page: Page):
    """NEG-GL-01: If the user denies location permission on the /geolocation page,
    page.goto(f"{BASE_URL}/geolocation")

    # Mock geolocation to deny permission
    page.context.set_geolocation_permission("denied")

    page.get_by_role("button", name="Where am I?").click()

    # Expect a message indicating denial or unavailability
    expect(page.locator("#error")).to_be_visible()
    expect(page.locator("#error")).to_contain_text("Geolocation error: User denied

    # Ensure no JS errors occurred
    errors = []
    page.on("console", lambda msg: errors.append(msg.text) if msg.type == "error"
    page.wait_for_load_state("networkidle")
    assert not errors, f"JavaScript errors found after denying geolocation: {error

def test_EDGE_GL_01_Geolocation(page: Page):
    """EDGE-GL-01: If location services are temporarily unavailable on the /geoloc
    page.goto(f"{BASE_URL}/geolocation")

    # Intercept geolocation API calls and return a specific error code (PositionEr
    # This requires using page.route to mock the Geolocation API.

    # Define a route handler to intercept the geolocation request
    def handle_geolocation_request(route):
        # Respond with a mock error for position unavailable

```

```

route.fulfill(
  status=200,
  content_type="application/json",
  body="""
  {
    "code": 2,
    "message": "Position unavailable"
  }
  """
)

```

This is a more advanced mocking technique. The site's JS directly calls `navigator.getCurrentPosition` directly, we need `page.evaluate\_handle` or `page.evaluate` to mock it. For this specific site, the error message "Geolocation error: Position unavailable" is displayed when `getCurrentPosition` fails with code 2.

Mock the `getCurrentPosition` method to return an error

```

page.add_init_script("""
  navigator.geolocation.getCurrentPosition = function(success, error) {
    error({
      code: 2,
      message: "Position unavailable"
    });
  };
""")

```

```

page.get_by_role("button", name="Where am I?").click()

```

Expect the error message to be displayed

```

expect(page.locator("#error")).to_be_visible()

```

```

expect(page.locator("#error")).to_contain_text("Geolocation error: Position un

```

Ensure no JS errors occurred

```

errors = []

```

```

page.on("console", lambda msg: errors.append(msg.text) if msg.type == "error"

```

```

page.wait_for_load_state("networkidle")

```

```

assert not errors, f"JavaScript errors found after simulating geolocation unav

```

```

def test_EDGE_WIN_01_Windows(page: Page):

```

```

    """EDGE-WIN-01: If pop-up blockers are enabled, a new window may not open from

```

```

    page.goto(f"{BASE_URL}/windows")

```

Configure Playwright's browser context to block popups.

```

# Playwright's `context.expect_page` with a short timeout can simulate this.
# If a new page is not opened within the timeout, it's effectively blocked.

# Set a short timeout for expecting a new page
page.context.set_default_navigation_timeout(5000) # Set a reasonable timeout f

# Attempt to click the link and expect a new page to *not* open within a short
with pytest.raises(TimeoutError): # Expect a TimeoutError if the page doesn't
    with page.context.expect_page(timeout=1000) as new_page_info: # Expect pag
        page.get_by_role("link", name="Click Here").click()
    # If it somehow opens, fail the test (this block should not be reached)
    new_page = new_page_info.value
    if new_page:
        pytest.fail("New window opened despite simulating pop-up blocker.")

# Ensure the base page remains functional
expect(page.get_by_role("heading", name="Opening a new window")).to_be_visible

# Ensure no JS errors occurred on the base page
errors = []
page.on("console", lambda msg: errors.append(msg.text) if msg.type == "error"
page.wait_for_load_state("networkidle")
assert not errors, f"JavaScript errors found on base page after blocked popup:

def test_EDGE_JE_01_JavaScript_Error(page: Page):
    """EDGE-JE-01: The JavaScript error on load on the /javascript_error page is i
    errors = []
    page.on("console", lambda msg: errors.append(msg.text) if msg.type == "error"

    page.goto(f"{BASE_URL}/javascript_error")
    page.wait_for_load_state("networkidle")

    # Assert that the intentional JS error is reported
    assert any("Cannot read properties of undefined (reading 'xyz')" in error for
        "Expected JavaScript error 'Cannot read properties of undefined (reading '

    # Assert that the page content is still rendered and accessible
    expect(page.get_by_role("heading", name="JavaScript error")).to_be_visible()
    expect(page.locator(".example")).to_be_visible()
    expect(page.locator(".example")).to_contain_text("This page has a JavaScript e

    # Attempt to interact with an element to show partial accessibility

```



Download final tests.py

```
# For example, check if the footer is still there  
expect(page.locator("#page-footer")).to_be_visible()
```